The Implementation Codex

Phase-Gated Specification Packages for AI-Assisted System Realization

The defensible thesis is that a sufficiently explicit, verified Codex may reduce executor discretion and implementation rework within a declared profile.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

AI-assisted software construction is highly sensitive to underspecified dependencies, interfaces, invariants, tooling, and acceptance criteria. The Implementation Codex is a proposed realization package that binds a system specification to dependency structure, build phases, reusable patterns, language and environment bindings, verification gates, authority, evidence, and a refinement journal.

The source reports eight sequential builds across Python and TypeScript using two model classes. Those builds are a valuable case study, but they do not yet demonstrate deterministic realization, executor or language independence, or that correctness resides in the specification. Codex versions, environments, test counts, and interventions changed; some table entries report final green status rather than first-pass status; the same models were not always evaluated on the same version; and zero new observations can mean saturation, weak observation, or both.

This revision treats implementation as a relation among specification, configuration, executor, tools, environment, realization, verification, and evidence. It defines a Codex envelope, distinguishes conformance from correctness and first-pass gate success, expands failure attribution, places the method at the IC-to-IR segment of the realization funnel, and specifies an independent held-out validation program. The defensible thesis is that a sufficiently explicit, verified Codex may reduce executor discretion and implementation rework within a declared profile.

Source, Evidence, and Standing

This is an AI-assisted integrated edition derived from one exact author manuscript. Its custody and comparison manifest is
`work/paper-saturation-v1/evidence/the-`

`implementation-codex/integration-custody-manifest.v0.1.json`,
SHA-256
`447864C5730A641F6041ABE5107005E5628308B5755BB24FF871507B7FABF8E7`.
The source remains unchanged.

The method, document classes, phases, journal, build narrative, findings, comparisons, and conclusions are author material. Formal qualifications, revised interpretation, corpus integration, evaluation, countertests, and decisions are AI editorial or synthetic candidates until adopted. The reported build results were not independently re-executed or audited in this pass.

1. What an Implementation Codex Is

An Implementation Codex is a versioned package for realizing a selected system configuration. Define:

$$K = (S, G, L, P, B, V, J, A, E),$$

where S is the specification and interfaces, G the dependency graph, L a phase plan or partial-order schedule, P reusable patterns, B language, tool, and environment bindings, V verification and acceptance profiles, J refinement journal and amendment lineage, A authority and decision boundaries, and E evidence and custody requirements.

The Codex is more than a prompt and less than an executable in itself. An executor still interprets it through a runtime, tools, libraries, environment, and available capabilities.

2. Realization Relation

Represent one build as:

$$X(K, e, r, c) \to (I, W, O),$$

where e is the executor, r runtime and toolchain, c build configuration, I the candidate implementation, W evidence, and O observations, interventions, failures, and unresolved remainder.

Identical Codex bytes do not guarantee identical output unless executor, dependencies, environment, scheduling, randomness, and external state are sufficiently closed. Independent implementations may also differ while remaining equivalent under an explicit conformance profile.

3. Six Core Document Classes, Expanded

The source's six classes remain useful:

1. **Dependency graph:** declared component, artifact, authority, and test dependencies.
2. **Phase linearization:** one permitted schedule through a dependency partial order.
3. **Phase specifications:** inputs, outputs, interfaces, invariants, failure behavior, and exit criteria.
4. **Patterns library:** versioned reusable constructions with preconditions and limits.
5. **Validation protocol:** tests, structural checks, oracles, evidence, and verdict rules.
6. **Binding guides:** language, library, operating-system, build, package, and runtime mappings.

The package also needs source and requirement custody, threat and failure models, effect boundaries, migration and rollback, unresolved decisions, and exact artifact manifests. Binding guides are part of the operative specification; their existence qualifies any claim that the system specification is language-independent.

4. Phase-Gated Construction

Phases can reduce error propagation by refusing continuation when prerequisites or evidence fail. The dependency graph may permit parallel, iterative, or conditional paths; a single strict sequence is one realization, not always structural necessity.

A gate should bind:

- exact inputs and predecessor evidence;
- obligations newly activated and inherited;
- required outputs and artifact identities;
- checks and oracles;
- allowed warnings, waivers, and unresolved items;
- failure and recovery behavior; and
- authority to accept or revise the phase.

Passing a gate supports declared conformance at that boundary. It does not prove absence of untested defects or correctness of the governing specification.

5. Conformance, Correctness, and First Pass

Keep separate:

- **Codex conformance:** the realization satisfies declared requirements under profile p.

- **test-suite success:** selected tests and checks pass in an identified environment.
- **first-pass gate success:** the initial unmodified realization attempt passes those gates.
- **functional correctness:** behavior satisfies the intended problem under relevant conditions.
- **system fitness:** the realization is safe, usable, secure, maintainable, and appropriate for its context.

Correctness is not a property of specification alone. It is a relation among problem, specification, realization, environment, observation, and acceptance. A perfectly realized wrong specification remains wrong.

6. Failure Attribution and Journal

Failures should not be assigned automatically to the specification. Classify at least:

- `SPECIFICATION_GAP`
- `SPECIFICATION_AMBIGUITY`
- `SPECIFICATION_CONFLICT`
- `BINDING_OR_ENVIRONMENT_GAP`
- `EXECUTOR_ERROR`
- `TOOL_OR_DEPENDENCY_FAILURE`
- `TEST_OR_ORACLE_DEFECT`
- `CUSTODY_OR_EXECUTION_DEFECT`
- `UNKNOWN_OR_MULTICAUSAL`
- `USABILITY_OR_PROCESS_FRICTION`
  Each journal entry binds evidence, affected version, reproduction, impact, provisional cause, alternatives, decision authority, amendment, and regression. The implementation may need an immediate bounded repair while the Codex is also revised. Prohibiting all implementation fixes can delay recovery or force speculative specification changes.

7. Source-Reported Case Study

The source reports eight builds and 53 cumulative observations. The table should be retained as a source report with normalized columns for attempted tests, initially executed tests, initial passes, post-intervention passes, halts, manual interventions, Codex changes, environment, model, language, and evidence location.

Several present entries are not directly comparable:

- Build 1 is listed as 178/178 GREEN while the narrative reports 169/178 first-pass and later fixes.
- Builds 4 and 5 report 0/0 and 0/145, which are execution or harness failures rather than correctness scores.

- Build 6 uses 196 tests, Build 7 uses 178, and Build 8 uses 145.
- Python model comparisons use different Codex versions.
- TypeScript v1.5 runs use the same Codex label but different test counts.
- an authoring/refinement model is not an independent blind executor.

These facts do not erase the case study. They change its standing from validation to an informative iterative-development record requiring custody review.

8. What Observation Decay Can Show

Declining new observations can indicate that a tested profile is becoming better specified. It can also reflect repeated tasks, narrower observation, reduced reviewer effort, shared blind spots, changed tests, or overfitting to the executors and suite.

Call a profile provisionally saturated only after repeated, independently observed executions of a frozen Codex yield low material change, while hidden tests, mutation, adversarial cases, new environments, and counterimplementations retain sensitivity. Zero journal entries in one run is not closure.

9. Cross-Executor and Cross-Language Claims

Cross-executor validation requires the same frozen Codex, task, environment class, input fixtures, acceptance profile, and hidden tests, with independent or blinded executors and no contamination from prior artifacts. Compare not only pass counts but semantic equivalence, structure, untested behavior, security, maintainability, and intervention.

Cross-language validation additionally requires a language-neutral contract and language-specific bindings. Success can support portability of selected obligations. It does not show that language is irrelevant: memory models, type systems, concurrency, package ecosystems, runtimes, and failure semantics may change the assurance argument.

The source results support a hypothesis that explicit binding guides reduce language-specific discretion; they do not yet prove that specification rather than model or language caused correctness.

10. AI-Executable Specification

"AI-executable" should mean that a named executor profile can consume the package and produce a candidate realization plus evidence without unscheduled open editorial questions beyond declared thresholds. It should not imply mechanical determinism or complete semantic understanding.

Useful criteria include requirement and dependency coverage, typed interfaces, explicit defaults, bounded choices, authority and effect boundaries, executable gates, diagnostic failures, closed or declared dependencies, environment reproducibility, and evidence-preserving output.

No specification can prohibit all undefined behavior merely by declaration. Unknowns and not-covered regions should be first-class.

11. Prompting and Codex Discipline

The comparison is not prompt versus non-prompt: a Codex still reaches a model through context, files, tools, or APIs. The substantive distinction is between an ephemeral instruction exchange and a versioned, source-bound, testable realization package with persistent artifacts, gates, evidence, and refinement lineage.

Prompt engineering can be structured and reproducible; a Codex can still be ambiguous or model-dependent. Compare workflows empirically rather than defining one as inherently governed.

12. Specification as Primary Artifact

The Codex can become the primary design and reconstruction artifact when it captures decisions more faithfully than any one implementation and can generate multiple conforming realizations. Source code remains an independent evidentiary and operational artifact; it may reveal platform facts, emergent behavior, security issues, or performance constraints absent from the specification.

Rebuild-from-specification, patch-and-reconcile, and migrate-with-proof are alternative maintenance strategies. Their choice depends on cost, urgency, state, data migration, compatibility, and confidence.

13. Corpus Integration and Authorial Compression

The Implementation Codex occupies the selected-configuration-to-realization boundary. **The Ableman Format Specification** defines artifact and stage roles. **HLIR** provides human-facing proposal representations. **MCIR** provides a canonical governed Intermediate Representation candidate. **Governance as Compilation** defines stage contracts, remainder, authority, and effect separation.

The Codex is therefore an **IC-to-IR realization package**, not the full constitutional stack. IC can mean Instance Constitution or Instance Configuration; IR can mean Instance Realization or Implementation Realization. Here IR is the realization stage, distinct from Intermediate Representation in MCIR and HLIR.

The **Governed Meaning Base (GMB)** can retain exact specifications, bindings, builds, journals, tests, decisions, and revisions. The universal **GSM** can type depends-on, realizes, conforms-under, failed-at, revised-by, and evidenced-by relations without turning them into correctness.

14. Realization Funnel

$$
\begin{aligned}
&\mathit{Possibility}\\
&\rightarrow \mathit{Specification}\\
&\rightarrow CM_1\\
&\rightarrow CM_2\\
&\rightarrow IC\\
&\rightarrow IR\\
&\rightarrow \mathit{Evidence}\\
&\rightarrow \mathit{Revision}
\end{aligned}
.
$$

- **Possibility:** enumerate implementation strategies, languages, executors, tools, schedules, and maintenance modes.
- **Specification:** define system obligations, interfaces, boundaries, and acceptance claims.
- **CM1:** retain admissible realization configurations, including costly or undesirable variants.
- **CM2:** prefer configurations by assurance, portability, simplicity, performance, cost, and maintainability.
- **IC:** select an Instance Constitution or Instance Configuration and its Codex package.
- **IR:** produce one Instance Realization or Implementation Realization through a declared executor and environment.
- **Evidence:** test conformance, correctness claims, security, portability, recovery, and fitness.
- **Revision:** amend specification, configuration, bindings, realization, or tests with preserved lineage.

This paper specifies a method spanning late Specification through Revision. It does not certify any source-reported IC or IR.

15. Validation Program

Freeze a Codex version and exact environment contract. Run multiple blinded executors, including models and non-model baselines where feasible, from clean state. Use public gates for construction and separately held-out acceptance, mutation, security, and fault tests. Preserve every intervention and failure.

Repeat across languages with equivalent claim inventories and normalized tests. Have independent reviewers inspect semantic equivalence, not just counts. Then test a new system not used in Codex refinement.

Measure first-pass completion, gate success, hidden-test success, requirement coverage, mutation sensitivity, cross-executor divergence, semantic equivalence, intervention count, token and time cost, journal yield, defect escape, recovery, rebuild reproducibility, and maintenance burden.

IC-to-IR Realization Package

Across the completed P1 corpus, the Implementation Codex is the **IC-to-IR realization package that binds a selected Instance Constitution / Instance Configuration to one or more candidate Instance Realizations / Implementation Realizations through dependencies, phases, bindings, gates, evidence, and refinement lineage**. It is not the full constitutional stack, a prompt replacement by definition, or proof that correctness resides in specification.

- **Adjacent work:** *The Ableman Format Specification (AFS)*
  Supplies: Artifact roles, profiles, canonicalization, validators, and conformance vectors.
  Implementation-Codex boundary: AFS can standardize Codex artifacts without deciding a system's correctness.
- **Adjacent work:** *Governance as Compilation*
  Supplies: Stage contracts, lowering, remainder, authority, effect, and receipts.
  Implementation-Codex boundary: The Codex packages one selected realization path through those stages.
- **Adjacent work:** *Admissibility Engineering*
  Supplies: Pathway inventories, constraint bindings, saturation gaps, and counterexample search.
  Implementation-Codex boundary: AE tests whether the Codex sufficiently constrains consequence-bearing realization routes.
- **Adjacent work:** *From CI/CD to ACG: From Tests to Courts in Agentic Software Development*
  Supplies: Governed construction, adjudication, promotion, and revision workflow.
  Implementation-Codex boundary: ACG can execute and review a Codex without becoming part of its specification identity.
- **Adjacent work:** *From Content-Addressed Storage to Governed Semantic Project Lineage*
  Supplies: Exact project heads, predecessors, projections, and reconstructable successors.
  Implementation-Codex boundary: Project lineage preserves Codex, build, journal, and implementation evolution.

A realization package should bind:

$$\begin{aligned} K = (&specification, instance_configuration, dependencies,\\ &phase_graph, patterns, bindings, executor, environment,.\\ &gates, tests, authority, evidence, journal, remainder) \end{aligned}$$

Conformance, test success, first-pass gate success, functional correctness, security, usability, and contextual fitness remain separate claims. Frozen-Codex replication requires exact environment and toolchain contracts, independent or blinded executors, held-out and mutation tests, and every intervention. Cross-language results require obligation-equivalence relations rather than artifact equality.

Observation decay supports saturation only when independent challenge remains sensitive. Zero new journal entries can also reflect weak observation, repeated tasks, shared blind spots, suppressed feedback, or overfitting. A Codex must retain unresolved decisions and implementation discoveries rather than forcing every discrepancy into speculative specification repair.

The universal GSM can type
`PACKAGES_INSTANCE_CONFIGURATION`,
`GUIDES_IMPLEMENTATION_REALIZATION`, `ORDERS_PHASE_BY_DEPENDENCY`,
`BINDS_LANGUAGE_ENVIRONMENT`, `PASSES_CONFORMANCE_GATE`, `FAILS_HELD_OUT_TEST`,
`ATTRIBUTES_REALIZATION_FAILURE`, `REVISES_CODEX_FROM_EVIDENCE`,
`LEAVES_IMPLEMENTATION_REMAINDER`,
and
`REBUILDS_FROM_FROZEN_PACKAGE`. These relations keep the IC-to-IR role explicit.

This integration establishes four editorial commitments: classify the Implementation Codex as the IC-to-IR realization package; adopt the full realization-package envelope; require separate conformance, correctness, security, fitness, first-pass, and saturation claims; and preserve implementation repair, held-out challenge, and unresolved remainder under exact lineage.

Proposal-to-Effect Realization Chain

The fixed-seed comparison with the Jnana open-source projection, *The Ableman Wheel*, HLIR, and the constitutional syscall architecture exposes the Implementation Codex's complete interface position:

$$\begin{aligned} R = (&HLIR_proposal, admitted_specification, IC, \\ &implementation_codex, IR, effect_boundary, evidence,. \\ &revision) \end{aligned}$$

HLIR is an Intermediate Representation on the human proposal and repair side. The Codex begins only after relevant ambiguity, authority, and configuration choices have been admitted or deliberately retained as bounded decisions. Its output guides an Instance Realization / Implementation Realization. These two uses of IR remain distinct even when both appear in one pipeline.

Jnana can govern proposal, admission, capability, and transformation stages, while the Codex packages the selected realization path. A constitutional syscall can mediate a downstream cyber-physical effect boundary. Neither interface is

mandatory: the Codex must declare the actual governance and effect mechanisms used and may not infer their coverage from architectural names.

The Ableman Wheel supplies a demanding domain profile for apparatus, calibration, analysis, safety, and run evidence. A Codex can improve traceability from selected configuration to apparatus and experiment without authorizing construction or establishing the anomaly. A realized artifact should retain every deviation from the frozen package and every implementation discovery that changes the specification or configuration claim.

The universal GSM can type
`LOWERS_HLIR_PROPOSAL_TO_ADMITTED_SPECIFICATION`,
`PACKAGES_SELECTED_IC`, `GUIDES_REALIZATION_STAGE_IR`,
`USES_INTERMEDIATE_REPRESENTATION_WITHOUT_`
`ACRONYM_COLLAPSE`,
`CONNECTS_TO_EFFECT_BOUNDARY`, `RECORDS_REALIZATION_DEVIATION`,
and
`DOES_NOT_AUTHORIZE_DOMAIN_EFFECT`.

This randomized review establishes four editorial commitments: adopt the full proposal-to-effect realization chain; enforce the two context-resolved IR senses; require named governance and effect-boundary interfaces rather than implied coverage; and retain domain authorization outside the Codex.

A Non-Authorizing Realization Contract

Ten downstream uses place the Implementation Codex between a selected IC and one or more realization-stage IRs. Admissibility Engineering tests whether its phases and gates cover the relevant realization pathways. AFS supplies artifact and profile roles. HLIR supplies human-reviewable proposals upstream. Jnana, Jnana II, and the open-source projection supply governance, transformation, and contribution contexts. Split-authority coordination supplies role boundaries. ACG supplies assurance lifecycle. The syscall and Ableman Wheel cases expose cyber-physical and apparatus effect ceilings.

The downstream-stable package is:

$$\begin{aligned} K = (&selected_IC, predecessors, dependencies, environment,\\ &bindings, phases, partial_order, gates, tests,\\ &held_out_challenges, authority, executor, journal,\\ &deviations, evidence, remainder, successor) \end{aligned}.$$

The Codex reduces and records implementation discretion; it does not eliminate discovery. A realization may reveal an omitted requirement, invalid dependency, impossible phase order, unsafe assumption, language-specific semantic change, or environment mismatch. Such findings must flow back as proposals to Specifi-

cation, CM1, CM2, or IC rather than being silently repaired in code or rewritten into the frozen package.

Conformance to the Codex, test passage, first-pass realization, correctness, security, fitness, reproducibility, and saturation are separate claims. Cross-executor or cross-language agreement requires an explicit equivalence relation and independent artifacts. A domain profile can use the Codex to bind parts, calibration, safety assumptions, and evidence without authorizing construction, operation, data access, human-subject involvement, environmental effect, or device dispatch.

The universal GSM can type
`PACKAGES_SELECTED_IC`, `GUIDES_REALIZATION_STAGE_IR`,
`RECORDS_IMPLEMENTATION_DEVIATION`, `RETURNS_DISCOVERY_TO_UPSTREAM_STAGE`,
`CONFORMS_WITHOUT_PROVING_CORRECTNESS`, `TESTS_HELD_OUT_CHALLENGE`,
and
`WITHHOLDS_DOMAIN_EFFECT_AUTHORITY`.

This reverse-dependency review establishes four editorial commitments: adopt the downstream-stable package; require bidirectional discovery and revision flow; keep conformance, tests, first pass, correctness, security, fitness, reproducibility, and saturation distinct; and preserve all domain-effect authority outside the Codex.

16. Counterevidence and Failure Modes

The method should be narrowed or rejected where: (1) final green status is called first-pass success; (2) changing Codex versions are treated as one experiment; (3) different test counts are compared directly; (4) tests that never ran count as failures or passes incorrectly; (5) executor involvement in authoring is called independent; (6) one run establishes reproducibility; (7) zero observations proves completeness; (8) observation processes weaken; (9) test suites overfit prior failures; (10) hidden tests fail; (11) mutation testing reveals weak gates; (12) the specification is wrong; (13) tests encode the same error; (14) passing tests imply system fitness; (15) conformance implies correctness; (16) correctness implies safety; (17) all failures are assigned to specification; (18) executor errors are rewritten as requirements; (19) tool failures are hidden; (20) implementation repairs are prohibited during urgent recovery; (21) phase order suppresses useful parallelism; (22) gates admit invalid intermediate state; (23) waivers become ambient; (24) binding guides omit runtime behavior; (25) languages change assurance properties; (26) packages or services drift; (27) artifacts are not exact; (28) rebuilds are not reproducible; (29) model versions drift; (30) prompts and tools remain hidden; (31) token or context limits truncate the Codex; (32) a lower-capability executor silently omits work; (33) code is treated as disposable despite state and migration; (34) the Codex cannot express discovered requirements; (35) GMB inclusion becomes validation; (36) GSM relations become correctness; (37) corpus roles are merged; (38) a case study is presented as general law; (39)

citations are inaccurate; (40) conventional formal or generative methods perform equally; (41) costs exceed reduced rework; or (42) external research contradicts novelty or causal claims.

A Better Specification Reduces Discretion; It Does Not Contain Correctness

Public orientation

The Implementation Codex packages a selected Instance Constitution or Instance Configuration into dependencies, environment, binding guides, phases, gates, tests, authority, journal, deviations, evidence, and successor rules for an Instance Realization or Implementation Realization. It makes implementation choices visible and revisable. It does not authorize construction or make a flawed specification correct.

Adversarial tests

1. **Specification-error attack.** Every executor conforms to the same omitted, unsafe, contradictory, or semantically wrong requirement.
2. **Gate-overfit attack.** The realization targets visible tests and prior failures while held-out, mutation, negative, and environmental challenges expose weakness.
3. **First-pass relabeling attack.** A repaired final candidate or changed Codex is reported as initial unmodified success.
4. **Executor-independence attack.** Multiple agents share the Codex author, model family, prompts, tools, dependencies, examples, or error and are counted as independent.
5. **Language-equivalence attack.** Cross-language builds are considered equivalent despite differing runtime, memory, concurrency, dependency, and security semantics.
6. **Discovery-suppression attack.** Implementation reveals a missing requirement or unsafe assumption but the executor silently fixes code rather than returning the issue upstream.
7. **Journal-blame attack.** Failure attribution is rewritten to protect the specification, executor, tool, or evaluator instead of preserving contested evidence.
8. **Codex-authority attack.** A complete package is treated as permission to construct, deploy, access data, operate a device, involve people, or create an environmental effect.

Claim and evidence ceiling

The paper proposes a non-authorizing realization contract that reduces and records implementation discretion. It does not establish specification correctness, complete requirements, first-pass performance, cross-executor independence, language independence, reproducibility, safety, security, fitness, saturation, or

effect authority. The source-reported build study remains source-reported until exact artifacts and procedures are audited.

Claims about program synthesis, refinement, model-based engineering, test adequacy, AI coding, empirical productivity, reproducible builds, or comparative workflow benefit require current primary sources and frozen-version repeated experiments. No external research was added in this pass.

Reader-facing consequence

The practical benefit is an implementation package that tells an agent what is fixed, what may vary, how to prove the result, and where to return discoveries - while keeping permission to build or deploy outside the package itself.

18. Conclusion

The Implementation Codex is a promising discipline for making selected implementation decisions explicit before an executor acts. Dependency structure, phase gates, patterns, binding guides, verification, and a journal can move ambiguity upstream and make refinement cumulative.

Its strongest defensible principle is not that correctness lives in the specification. It is that implementation discretion can be made visible, reduced, tested, and revised through a versioned realization contract. Whether that contract is complete enough, correct enough, portable enough, and economical enough remains an empirical question for each profile.

Reference and Verification Note

No external source was searched in this pass, and citations inherited from the source were not independently verified. Further evidentiary strengthening requires authorized primary-source review of formal specification, design by contract, program synthesis and refinement, model-based engineering, reproducible builds, software product lines, test adequacy and mutation, AI code generation, multi-agent software development, cross-language conformance, and empirical software engineering.

Realization Scope and Evidence Boundary

This successor makes the paper's stage boundary explicit. It narrows claims rather than manufacturing missing evidence: a stage named as context is not thereby claimed as implemented, validated, or complete.

- **Instance Configuration.** The present paper does not claim a complete instance configuration treatment. That stage remains successor work and cannot be inferred from the architecture described here.

- **Possibility Space.** Existing source passages use this stage only as bounded context or an obligation. The paper does not claim an independently validated contribution at this stage.

Corpus-Wide Reference Positioning

This paper argues that aI-assisted software construction is highly sensitive to underspecified dependencies, interfaces, invariants, tooling, and acceptance criteria. Its nearest external comparison fields are provenance, formal, open sovereign. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: A complete provenance graph can faithfully record a false or unauthorized process; provenance is evidence about lineage, not automatic truth or legitimacy. Formal assurance remains relative to the encoded model, trusted base, and refinement chain; proof does not remove specification error or unmodeled environmental behavior. Source availability and reproducible builds improve inspectability but do not alone guarantee comprehensibility, practical exit, hardware control, or sustainable maintenance. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] W3C. *PROV-O: The PROV Ontology*. W3C Recommendation, 30 April 2013. https://www.w3.org/TR/prov-o/ **Relationship:** Adjacent provenance standard; it models entities, activities, and agents, but does not by itself establish the paper's stronger authority or standing claims.

[**E2**] Santiago Torres-Arias et al. "in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes." *28th USENIX Security Symposium*, 2019, pp. 1393-1410. usenix.org source **Relationship:** Predecessor supply-chain lineage system used for comparison; no influence claim is made unless the source head already documents one.

[**E3**] OpenSSF. *Supply-chain Levels for Software Artifacts (SLSA), Version 1.2*. 2026. https://slsa.dev/spec/v1.2/ **Relationship:** Adjacent software-supply-chain assurance standard, especially relevant to provenance and verification boundaries.

[**E4**] OpenTelemetry Authors. *OpenTelemetry Specification*. https://opentelemetry.io/docs/specs/otel/ **Relationship:** Adjacent observability standard for traces, metrics, logs, and context propagation; telemetry is not equated with adjudicated evidence.

[**E5**] George C. Necula. "Proof-Carrying Code." *POPL '97*, 1997, pp. 106-119. DOI: 10.1145/263699.263712. https://doi.org/10.1145/263699.263712 **Relationship:** Predecessor for binding machine-checkable evidence to executable artifacts; the present paper may demand broader semantic or constitutional obligations.

[**E6**] Xavier Leroy. "Formal Verification of a Realistic Compiler." *Communications of the ACM* 52(7), 2009, pp. 107-115. DOI: 10.1145/1538788.1538814. https://doi.org/10.1145/1538788.1538814 **Relationship:** Predecessor verified-compilation result used to distinguish proven translation preservation from broader governance claims.

[**E7**] Patrick Cousot and Radhia Cousot. "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints." *POPL '77*, 1977. DOI: 10.1145/512950.512973. https://doi.org/10.1145/512950.512973 **Relationship:** Predecessor formal framework for sound approximation and fixpoint reasoning.

Ableman corpus

[**A1**] Adam Ableman. *Reasoning by Way of Receipts*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-04E427F6F4D94454`;
source SHA-256
`AF86A66CCD6A103DD75208500AE99148911A6453B6DFDF36A954279F21AB8489`.

Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *The Epistemic Lineage Engine.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-E0EC3DC7EABF5060`;
source SHA-256
`CDD74D9AFAB6532645E68F7F6C9F0F7A9E472D9B8E717290522566FF8DC01F11`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *The Three Hash Rule.* Ableman Corpus current edition; work identity `PAPER.PUB.20788617`;
source SHA-256
`F6FA73D083B70711A8C817F7779751D1506FEBA0E356DB9805BB1570663731B5`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *A Formal Theory of Work Custody.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-C8F2F9BF38C45E29`;
source SHA-256
`A80657E21BA277ADF1C229A2AAE221963B97A62EA2AF3EBC36434A916A8EEA44`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Governance Witnesses.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-9E8DA0EFE860DE5D`;
source SHA-256
`8BF8C3646040F6E1F5222A8AA92C1106AA1F841E8A2C65402F468C83D59D06E5`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *MCIR: A Canonical Intermediate Representation for Governed Transformation.* Ableman Corpus current edition; work identity `PAPER.PUB.21095303`;
source SHA-256
`EA129ECBA95F7B07CAA7B33A54F91F52913C94A14DA227EB9A06CFD083E4E5BF`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Governed Semantic Compilation.* Ableman Corpus current edition; work identity `PAPER.PUB.18453681`;
source SHA-256
`5FF6C636550B04DB0E2F6F94C379BB75AA0BD8781E19B047D7D3BB6D49071AC2`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

The entries below are corpus-internal comparison and dependency sources. Their inclusion does not establish external validation of this paper's empirical, safety, novelty, or performance claims.

Adam Ableman *A Theory of Agentic Observation.* Zenodo, 2026. doi:10.5281/zenodo.20437813

Adam Ableman *CARA: Constrained Artificial Reasoning Architecture.* Zenodo, 2025. doi:10.5281/zenodo.18099715

Adam Ableman *Constitutional Computation.* Zenodo, 2026. doi:10.5281/zenodo.20624488

Adam Ableman *Constraint-Bearing Composability.* Zenodo, 2026. doi:10.5281/zenodo.21632061

Adam Ableman *Fractal Governance Architecture.* Zenodo, 2026. doi:10.5281/zenodo.20754189

Adam Ableman *Governed Semantic Compilation: Lawful Continuation, Refusal Semantics, and Epistemic Promotion in Large Language Models.* Zenodo, 2026. doi:10.5281/zenodo.18453681

Adam Ableman *MCIR: A Canonical Intermediate Representation for Governed Transformation.* Zenodo, 2026. doi:10.5281/zenodo.21095303

Adam Ableman *Semantic Coordinate Identity Tokenization (SCIT).* Zenodo, 2026. doi:10.5281/zenodo.20532070

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus

License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.23848350